

# MEMORIA DEL PROYECTO INTERMODULAR

## WIKIVOLLEY

Autor: Edgar F. Ariza Carreño  
Curso: 1º CFGS en Desarrollo de Aplicaciones Multiplataforma (DAM)  
Centro: Universidad Alfonso X el Sabio (UAX)  
Enero — Junio 2026

## 1. Introducción y objetivo del proyecto

La idea de WIKIVOLLEY surgió porque quería hacer algo que tuviera sentido más allá del proyecto en sí. Llevo años ligado al voleibol y me parecía un ámbito perfecto para aplicar lo que he ido aprendiendo a lo largo del curso: hay contenido real que gestionar, hay usuarios con necesidades concretas y, sobre todo, hay margen para montar algo que funcione de verdad.

El objetivo principal era demostrar que soy capaz de unir distintas piezas — programación web, bases de datos, administración de sistemas y algo de automatización— en un producto coherente. No quería entregar cuatro cosas sueltas, sino una sola plataforma donde todo tuviera su sitio y su razón de ser.

El punto de partida fue una página estática muy básica. A partir de ahí fui añadiendo capas: primero la base de datos y el backend, luego el sistema de usuarios con cifrado de contraseñas, después el control de roles y finalmente la automatización desde Python. Cada fase resolvía un problema concreto y encajaba con la siguiente.

## 2. Arquitectura de datos y entorno de sistemas

### 2.1. Base de datos relacional (MySQL)

La base de datos corre en local sobre XAMPP con MySQL y se llama 4736486\_volley. Ese prefijo numérico no es casualidad: reproduce exactamente cómo funciona el panel de control de AwardSpace, el servicio de hosting donde está desplegada la versión en producción. Quería que el entorno de desarrollo fuera lo más fiel posible a lo real para no tener sorpresas al subir el proyecto.

La estructura tiene cuatro tablas principales:

- bolsa\_empleo: almacena las ofertas de trabajo.
- cursos: catálogo de formación disponible.
- usuarios: registros de cuentas con rol y contraseña cifrada.
- postulaciones: tabla intermedia que relaciona usuarios con ofertas.

### 2.2. Conexión e integración de datos (conexion.php)

Todo pasa por un archivo central: conexion.php. Abre la conexión a MySQL usando la extensión MySQLi de PHP, configura el charset a UTF-8 para que las tildes no den problemas y, si algo falla al conectar, lo corta en seco con un mensaje claro. El resto de archivos simplemente hacen un include de este módulo, así que cambiar los datos de conexión en un solo sitio afecta a todo el proyecto instantáneamente.

## 3. Diseño y capa pública (CSS)

La parte visual está toda concentrada en styles.css. Opté por un diseño en modo oscuro porque encaja bien con la estética del vóley playa profesional y, siendo sincero, también porque es más cómodo de ver durante horas. El layout es responsivo, funciona bien tanto en escritorio como en móvil.

Las secciones públicas principales son tres:

- Noticias: se leen desde un archivo noticias.json en el backend y se pintan en una cuadrícula de tarjetas dinámicas. El contenido se puede actualizar sin tocar el HTML.
- Bolsa de empleo: consulta directa a MySQL. PHP recorre el resultado con un bucle while y transforma cada fila en una tarjeta visual con la información de la oferta.
- Cursos: mismo mecanismo que la bolsa de empleo, conectado a su propia tabla en la base de datos.

## 4. Sistema de seguridad, registro y roles

Este fue el apartado que más me costó arrancar pero también el que más satisfacción me dio cuando empezó a funcionar. Transformar una web pública en un entorno con acceso controlado tiene su miga, y hubo que pensar bien cada paso para que no hubiera agujeros.

### 4.1. Cifrado y registro de usuarios (registro.php)

Cualquier visitante puede crear una cuenta desde la pantalla de registro. Antes de guardar nada, el sistema comprueba que el correo no esté ya en la base de datos para evitar duplicados. La contraseña nunca se guarda en texto plano: se pasa por `password_hash()` de PHP, que genera un hash con `bcrypt`. Aunque alguien consiguiera volcarse la tabla de usuarios, lo que vería sería completamente inútil.

### 4.2. Control de acceso y privacidad (login.php)

El proceso de login comprueba el hash con `password_verify()` y, si cuadra, mete los datos clave del usuario en la sesión PHP: su ID, nombre y rol. A partir de ahí, el sitio sabe quién es en cada página.

Hay un detalle que me pareció importante: si alguien navega sin identificarse, los botones de «Aplicar» de la bolsa de empleo están bloqueados. La razón es concreta —proteger los correos de las empresas colaboradoras del spam automático— y la solución es sencilla pero efectiva.

## 5. El entorno de administración

Si el usuario que inicia sesión tiene el rol Admin en la base de datos, `index.php` renderiza un bloque de gestión avanzada que no existe para el resto de usuarios. No es que esté oculto visualmente: directamente no se carga en el HTML que llega al navegador. Eso me pareció lo correcto desde el punto de vista de seguridad.

El panel de admin tiene un cabecero diferenciado —fondo degradado, indicador de «ENTORNO RESTRINGIDO» y una barra tricolor— para que quede visualmente claro dónde empieza la zona de gestión.

### 5.1. Automatización híbrida PHP + Python

Desde el panel de admin hay un botón que dispara `ejecutar_script.php`. Este archivo usa `shell_exec()` para llamar al sistema operativo y ejecutar en segundo plano el script de Python `import_json.py`. Ese script hace tres cosas: regenera el archivo `noticias.json` con las noticias actualizadas, vacía y vuelve a rellenar la tabla de

bolsa\_empleo en MySQL, y hace lo mismo con la tabla de cursos. Todo de forma desatendida, en un solo clic.

Me gustó mucho este punto porque conecta el mundo web con la administración de sistemas de forma muy directa. PHP activa Python, Python escribe en ficheros y en la base de datos, y el resultado se refleja instantáneamente en la web pública. Es el tipo de automatización que se usa en entornos reales y aquí queda implementada de forma funcional.

## 5.2. Auditoría de interacciones con INNER JOIN

Para que el administrador pueda ver qué está pasando en la plataforma, hay un panel de analítica que cruza tres tablas —postulaciones, usuarios y bolsa\_empleo— mediante un INNER JOIN triple. El resultado es una tabla en tiempo real que muestra qué usuario ha hecho clic en qué oferta y cuándo. Nada del otro mundo en términos de SQL, pero muy útil en la práctica.

## 5.3. Gestión de cuentas y permisos

El último módulo del panel es una herramienta CRUD conectada a `modificar_usuario.php`. Muestra la lista completa de usuarios registrados y permite, desde la misma interfaz, ascender a alguien al rol de Admin, rebajarle los permisos si ya no corresponde, o eliminar directamente la cuenta si hace falta. Para este último caso, hay una confirmación en JavaScript para evitar borrados por accidente.

El sistema tiene una protección importante: ninguna acción puede aplicarse sobre la propia cuenta del administrador que está usando el panel. Eso evita que alguien se quede sin acceso por error.

## 6. Conclusión

WIKIVOLLEY ha sido el proyecto más complejo que he desarrollado hasta ahora, y también el más satisfactorio. No por lo que es en sí, sino por lo que ha supuesto: tener que tomar decisiones reales de arquitectura, resolver problemas que no estaban en los apuntes y hacer que piezas muy distintas trabajasen juntas.

He aplicado cosas de casi todos los módulos del curso: PHP y HTML/CSS del desarrollo web, MySQL de bases de datos, Python de programación, `shell_exec()` y la gestión del servidor de sistemas. Ver cómo encajan todas en un mismo producto es, creo, lo que justifica llamarlo proyecto intermodular.

Queda margen de mejora obvio: se podría sustituir el JSON plano por una API REST, añadir paginación a las tablas del admin, o montar un sistema de notificaciones por correo para las postulaciones. Pero para un primer año de DAM, estoy bastante contento con el resultado.